

метрики

Отличный вопрос! Давайте разберем ключевые метрики для задач ранжирования и классификации/регрессии, сделав особый акцент на NDCG.

I. Метрики для оценки ранжирования (Ranking Metrics)

Эти метрики критически важны для поисковых систем, рекомендательных систем и любых задач, где важен порядок выдачи объектов.

1. Precision@k и Recall@k — простые и интуитивные

- **Precision@k (Точность на k):** Доля *релевантных* объектов среди первых k рекомендованных (или выданных).
 - $\text{Precision@k} = (\# \text{ релевантных среди top-}k) / k$
 - **Интерпретация:** "Какой процент из показанных мной первых k элементов — хороший?"
 - Зависит только от k , не учитывает позицию внутри списка. Элемент на 1-м и на k -м месте вносят одинаковый вклад.
- **Recall@k (Полнота на k):** Доля *всех релевантных* объектов, которую удалось "отловить" в первых k рекомендациях.
 - $\text{Recall@k} = (\# \text{ релевантных среди top-}k) / (\text{общее } \# \text{ релевантных объектов})$
 - **Интерпретация:** "Какой процент от всех хороших элементов я показал в первых k ?"
 - Сильно зависит от общего числа релевантных объектов.

Пример (k=3): Система выдала [релевантный, нерелевантный, релевантный]. Всего релевантных объектов в базе 10.

- $\text{Precision@3} = 2/3 \approx 0.67$
- $\text{Recall@3} = 2/10 = 0.2$

2. MAP@k (Mean Average Precision) — учитывает порядок внутри top-k

Более продвинутая метрика, которая усредняет Precision не только по k , но и по всем релевантным объектам.

- **Average Precision (AP@k):** Для одного запроса/пользователя. Вычисляется как среднее значение $\text{Precision@}m$ в тех точках m (позициях), где был найден релевантный объект.

- $AP@k = (1 / \min(k, \text{\#релевантных})) \sum_{m=1..k} Precision@m$ (m-й объект релевантен)
- **Суть:** Штрафует за то, что релевантные объекты находятся не в начале списка.
- **MAP@k (Mean Average Precision):** Просто среднее значение $AP@k$ по всем запросам/пользователям в тестовой выборке.
 - **Интерпретация:** "Среднее качество ранжирования по всем запросам, с учетом позиций релевантных объектов в top-k".

Сильная сторона: MAP отлично подходит для бинарной релевантности (да/нет). Она напрямую оптимизирует цель "показать все релевантные объекты как можно выше".

3. NDCG@k (Normalized Discounted Cumulative Gain) — король соревнований. Почему?

NDCG — это наиболее популярная и мощная метрика для ранжирования, особенно в рекомендациях. Она решает ключевые ограничения предыдущих метрик.

А. Основные идеи и компоненты:

1. **Gain (Прирост G):** У каждого объекта есть **степень релевантности** (не просто 0/1, а, например, 0, 1, 2, 3, 4, 5). $G = 2^{\{\text{релевантность}\}} - 1$ (или просто сама релевантность). Чем выше релевантность, тем больше "ценность" объекта.
2. **Discounted (Дисконтированный):** Прирост на позиции i **дисконтируется** в зависимости от позиции: $G / \log_2(i + 1)$. Чем ниже в списке (больше i), тем сильнее штраф. Это отражает реальное поведение пользователей, которые смотрят в первую очередь на верхние позиции.
 - Штраф за позицию 2: $1 / \log_2(3) \approx 0.63$
 - Штраф за позицию 10: $1 / \log_2(11) \approx 0.29$
3. **Cumulative Gain (CG@k):** Простая сумма приростов первых k объектов. Не учитывает позицию.
 - $CG@k = \sum_{i=1..k} G_i$
4. **Discounted Cumulative Gain (DCG@k):** Сумма **дисконтированных** приростов. Учитывает и релевантность, и позицию.
 - $$DCG@k = \sum_{i=1..k} (2^{rel_i} - 1) / \log_2(i + 1)$$

(наиболее популярная формула).
5. **Normalized (Нормализованный):** DCG может быть каким угодно. Чтобы метрика была в диапазоне $[0, 1]$ и сравнима между разными запросами, ее **нормализуют** на **идеальный DCG (IDCG)**.

- $IDCG@k$ — это максимально возможный $DCG@k$, который достигается при **идеальной сортировке** объектов (самые релевантные — на самых верхних позициях).
- **Итоговая формула:** $NDCG@k = DCG@k / IDCG@k$

В. Пример:

Ранжирование: $[3, 2, 1, 0]$ (где числа — релевантности). $k=4$.

1. **Считаем $DCG@4$:** $(2^3-1)/\log_2(2) + (2^2-1)/\log_2(3) + (2^1-1)/\log_2(4) + 0 = 7/1 + 3/1.585 + 1/2 \approx 7 + 1.893 + 0.5 = 9.393$
2. **Считаем $IDCG@4$:** Идеальный порядок такой же ($[3, 2, 1, 0]$), поэтому $IDCG = 9.393$.
3. **Считаем $NDCG@4$:** $9.393 / 9.393 = 1.0$ (идеальное ранжирование).

Если бы порядок был $[1, 3, 2, 0]$:

- $DCG = (1/1) + (7/1.585) + (3/2) + 0 = 1 + 4.416 + 1.5 = 6.916$
- $IDCG = 9.393$ (не изменился)
- $NDCG = 6.916 / 9.393 \approx 0.736$

С. Почему NDCG — король?

1. **Учитывает градуальную релевантность.** Ошибка в ранжировании сильно релевантного объекта ($5 \rightarrow 1$) штрафует сильнее, чем слабо релевантного ($2 \rightarrow 1$).
2. **Учитывает позицию.** Разместить релевантный объект на 1-м месте гораздо ценнее, чем на 10-м (дисконтирование).
3. **Нормализован.** Позволяет корректно усреднять результаты по разным запросам/пользователям, у которых разное количество релевантных объектов.
4. **Интуитивно интерпретируем.** $NDCG=0.8$ означает, что качество ранжирования составляет 80% от идеального.

Итог по ранжированию: NDCG — наиболее полная и адекватная метрика для реальных задач, где важны и ранги, и "степень хорошеи". **MAP** — сильный конкурент для случая бинарной релевантности. **Precision/Recall** — хороши для быстрой оценки и понимания.

II. Метрики для классификации и регрессии

Эти метрики используются, когда задача — предсказать точное значение или класс, а не порядок.

Для регрессии (предсказание числа):

- **MAE (Mean Absolute Error, Средняя абсолютная ошибка):**

$$MAE = (1/N) * \sum |y_i - \hat{y}_i|$$

- **Плюс:** Легко интерпретируется ("средняя ошибка в единицах целевой переменной").
- **Минус:** Не штрафует сильнее за большие выбросы.
- **RMSE (Root Mean Squared Error, Среднеквадратичная ошибка):**

$$RMSE = \sqrt{(1/N) * \sum (y_i - \hat{y}_i)^2}$$

- **Плюс:** Сильнее штрафует за крупные ошибки (за счет квадрата). Математически удобна.
- **Минус:** Менее интерпретируема, чувствительна к выбросам.
- **Выбор:** Если крупные ошибки критичны — **RMSE**. Если все ошибки равнозначны — **MAE**.

Для бинарной классификации (вероятность/класс):

- **LogLoss (Logistic Loss, Логистическая ошибка):**

$$LogLoss = -(1/N) * \sum [y_i * \log(\hat{y}_i) + (1 - y_i) * \log(1 - \hat{y}_i)]$$

- **Суть:** Оценивает *качество предсказанных вероятностей*. Штрафует тем сильнее, чем больше уверенность модели в неверном ответе (например, прогноз $\hat{y}=0.99$ при истинном $y=0$ даст огромный штраф).
- **Король** для задач, где важна именно **калиброванная вероятность** (например, кредитный скоринг).
- **ROC-AUC (Area Under ROC Curve):**
 - **ROC-кривая** строится как зависимость **TPR (Recall)** от **FPR** при изменении порога бинаризации вероятностей.
 - **AUC** — площадь под этой кривой. Интерпретируется как **вероятность того, что случайно выбранный положительный объект будет иметь предсказанную вероятность выше, чем случайно выбранный отрицательный**.
 - **Преимущества:**
 1. Инвариантна к соотношению классов в выборке (несбалансированность не влияет).
 2. Оценивает качество **ранжирования** объектов по вероятности положительного класса (похоже на метрику ранжирования!).
 - **Когда использовать:** Когда важно, чтобы "хорошие" объекты в среднем имели оценку выше, чем "плохие" (задача ранжирования по вероятности). Менее чувствительна к конкретным порогам, чем ассигасу или F1-score.

Краткий итог-путеводитель:

- Оцениваешь порядок выдачи (поиск, рекомендации)? → **NDCG@k** (лучший), **MAP@k** (хорош для бинарного случая).
- Оцениваешь точность предсказания числа? → **RMSE** (большие ошибки важны), **MAE** (все ошибки равны).
- Оцениваешь качество предсказанных вероятностей? → **LogLoss**.
- Оцениваешь, насколько хорошо модель ранжирует объекты по вероятности класса (например, отклик на рекламу)? → **ROC-AUC**.